

Design Modelling

Team ArogyaSetu+

Group Number	14
Team Name	DHRUV-01

Team Members

Roll No.	PRN	Name
58	202301040064	Prasad Ingole
126	202301040140	Abhinav Kishor Shingote
127	202301040141	Netra Shreepal Rajmane
211	202301040242	Disha Samir Uplanchiwar
250	202301040283	Atharva Suryawanshi
299	202402040032	Yash Chopade

1.1 System Architecture Design

The proposed AI-Enabled Telehealth and Electronic Health Record system follows a layered architecture designed to ensure scalability, modularity, and maintainability. Each layer has well-defined responsibilities and communicates through clearly specified interfaces.

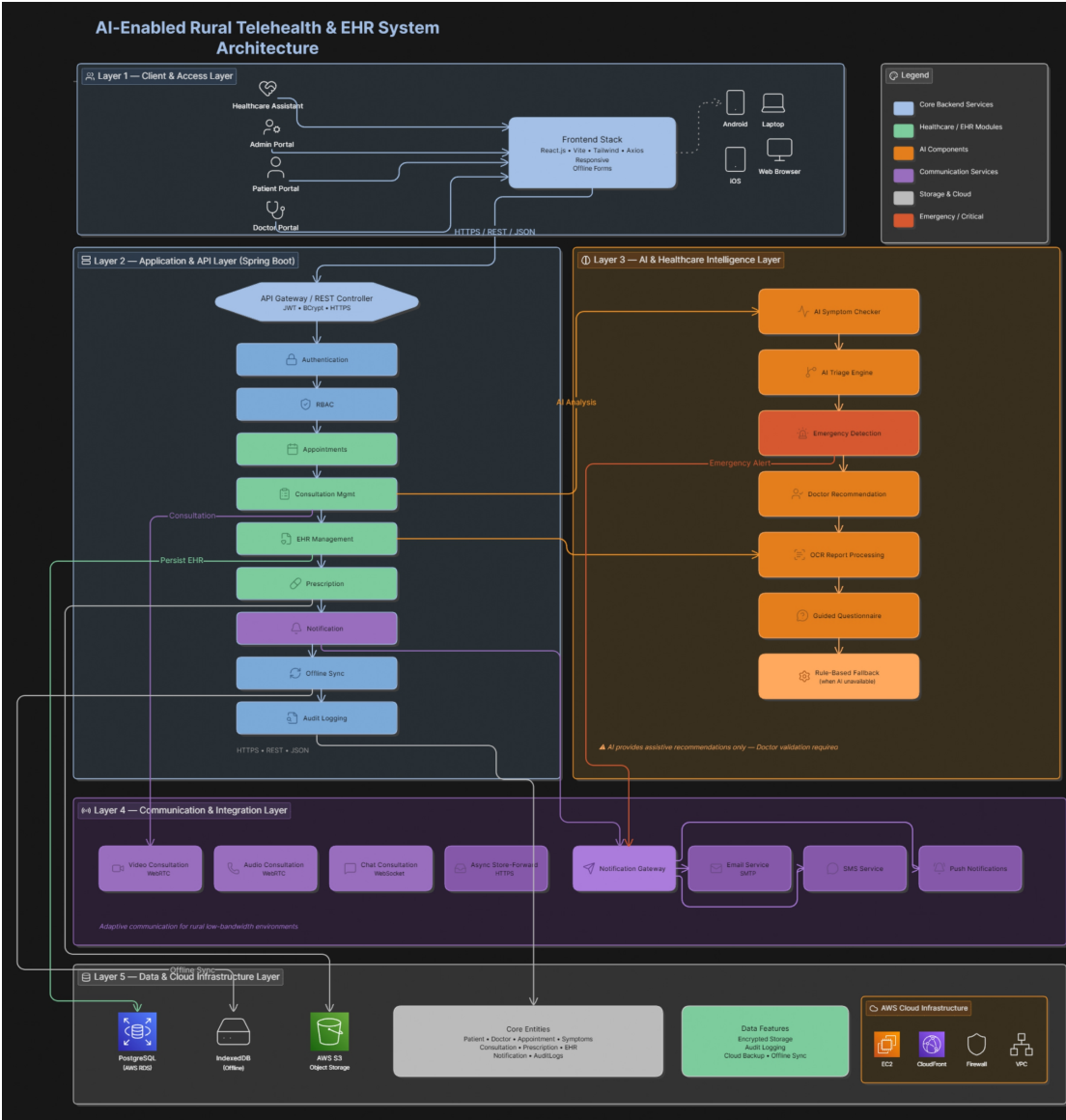


Figure 5.1: System Architecture Diagram

The architecture is composed of four principal layers:

1. **Presentation Layer** — React-based frontend interfaces for patients, doctors, and administrators.
2. **Application Layer** — Spring Boot microservices housing business logic and API endpoints.
3. **Integration Layer** — Bridges external services such as AI engines, WebRTC, and notification gateways.
4. **Data Layer** — PostgreSQL on AWS RDS plus Redis caching and IndexedDB for offline support.

1.2 Module Design

The system is decomposed into ten functional modules, each encapsulating a distinct responsibility to promote separation of concerns and ease of testing.

Table 1.1: System Module Overview

ID	Module Name	Responsibility
M1	Authentication Module	JWT-based authentication and role-based access control (RBAC)
M2	Patient Module	Patient profile management and symptom submission
M3	Doctor Module	Consultation management and patient record access
M4	Appointment Module	Appointment scheduling and calendar management
M5	EHR Module	Structured electronic medical record storage and retrieval
M6	AI Triage Module	Symptom analysis and priority classification
M7	Consultation Module	Real-time video, audio, and chat-based consultations
M8	Notification Module	Alerts, reminders, and push notifications
M9	Offline Sync Module	Offline-first support via IndexedDB synchronisation
M10	Admin Module	System monitoring, reporting, and user management

1.3 Data Flow Design

The figure below illustrates the end-to-end journey of a patient interaction through the system, from symptom entry to permanent EHR storage.

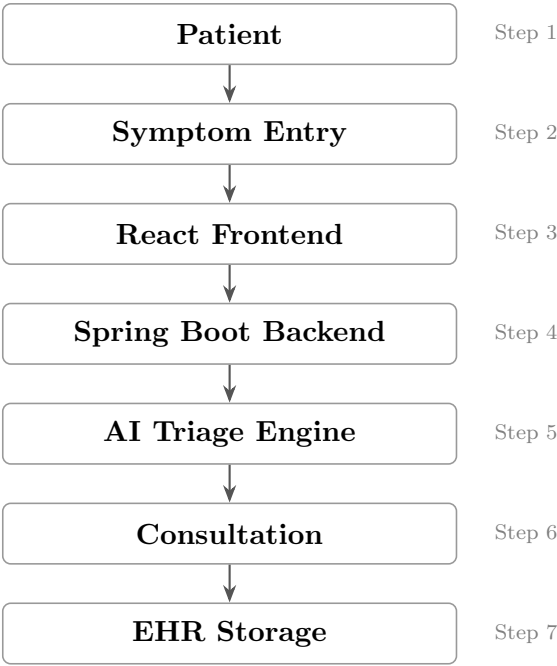


Figure 1.1: End-to-End Data Flow

1.4 Database Design (ER Model)

The system uses PostgreSQL (AWS RDS) as the primary relational database. It stores healthcare records, appointments, consultations, prescriptions, Electronic Health Records, notifications, and audit information.

1.4.1 Main Database Entities

Table 1.2: Database Entities and Purposes

Entity	Purpose
Patient	Stores patient profile and demographic information
Doctor	Stores doctor details, specialisation, and availability
Appointment	Stores consultation scheduling and status information
Symptoms	Stores patient-reported symptoms and AI-assigned priority
Consultation	Stores consultation history, notes, and diagnosis
Prescription	Stores prescribed medicine details and dosage
EHR	Stores structured electronic medical records
Notification	Stores system-generated alerts and reminders
AuditLogs	Stores security events and access tracking information

1.4.2 EHR Structure

The Electronic Health Record is structured as a composite document comprising six functional components, each capturing a distinct aspect of patient health information.

Table 1.3: EHR Component Structure

Component	Description
Patient Profile	Demographic information including name, age, blood group, and emergency contact
Symptoms	Patient-reported complaints and AI-assigned triage priority
Reports	Uploaded diagnostic reports, lab results, and scanned documents
Consultation Notes	Doctor observations, clinical findings, and diagnosis records
Prescriptions	Prescribed medicines, dosage instructions, and duration
Follow-up	Scheduled future consultations and monitoring instructions

1.4.3 Entity Relationships

Table 1.4: Entity Relationship Mappings

Relationship	Cardinality
Patient → Appointment	One-to-Many
Doctor → Appointment	One-to-Many
Patient → Symptoms	One-to-Many
Appointment → Consultation	One-to-One
Consultation → Prescription	One-to-One
Patient → EHR	One-to-One
Patient → Notification	One-to-Many

1.4.4 Entity Attribute Design

Patient Entity

Attribute	Type	Description
patient_id	BIGSERIAL	Primary Key
first_name	VARCHAR(50)	Patient first name
email	VARCHAR(100)	Email address
phone	VARCHAR(15)	Contact number
gender	VARCHAR(10)	Gender
blood_group	VARCHAR(10)	Blood group
address	TEXT	Patient address
emergency_contact	VARCHAR(15)	Emergency contact number
created_at	TIMESTAMP	Record creation date

Doctor Entity

Attribute	Type	Description
doctor_id	BIGSERIAL	Primary Key
full_name	VARCHAR(100)	Doctor full name
specialization	VARCHAR(100)	Medical department / speciality
experience	INTEGER	Years of clinical experience
email	VARCHAR(100)	Email address
availability	BOOLEAN	Current availability status

Appointment Entity

Attribute	Type	Description
appointment_id	BIGSERIAL	Primary Key
patient_id	BIGINT	Foreign Key → Patient
doctor_id	BIGINT	Foreign Key → Doctor
appointment_date	DATE	Scheduled consultation date
consultation_mode	VARCHAR(20)	Video / Audio / Chat
status	VARCHAR(20)	Scheduled / Completed / Cancelled

The entity relationship structure is summarised in Figure 1.2.

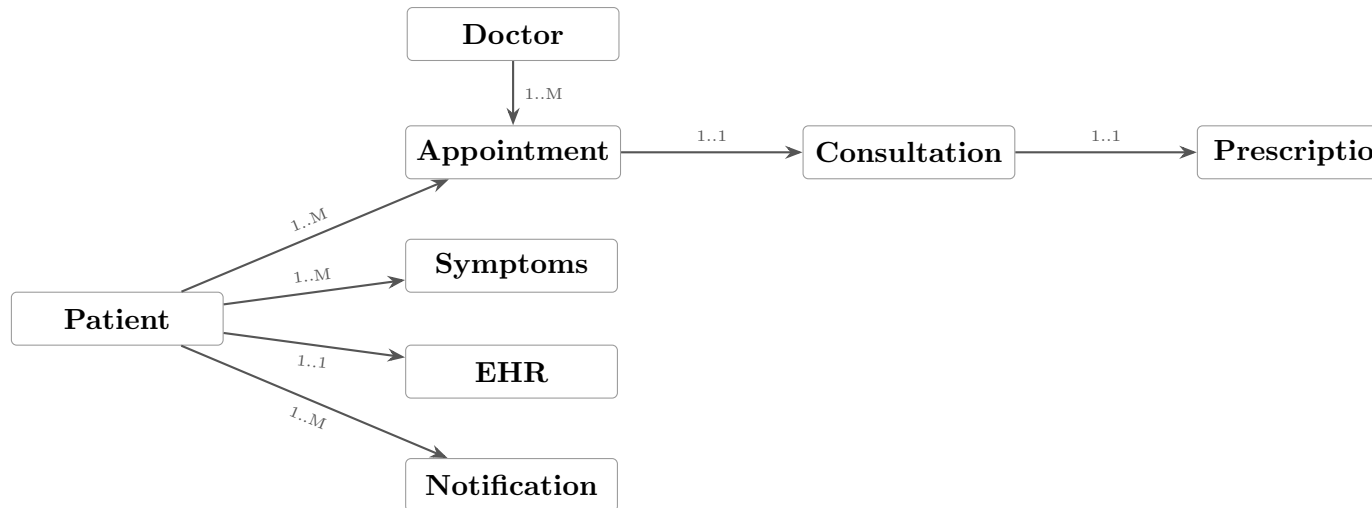


Figure 1.2: Entity Relationship Overview

1.5 Backend API Design

The backend follows REST architecture implemented using Spring Boot. All endpoints are versioned under the base URL `/api/v1`.

1.5.1 Authentication APIs

Base path: `/api/v1/auth`

Table 1.5: Authentication API Endpoints

Method	Endpoint	Purpose
POST	<code>/register</code>	New user registration
POST	<code>/login</code>	User authentication and token issuance
POST	<code>/forgot-password</code>	Initiate password reset flow
POST	<code>/logout</code>	Session termination
GET	<code>/profile</code>	Retrieve authenticated user profile

1.5.2 Patient APIs

Base path: `/api/v1/patient`

Table 1.6: Patient API Endpoints

Method	Endpoint	Purpose
GET	/profile	View patient profile
PUT	/profile	Update patient profile
POST	/symptoms	Submit new symptom report
POST	/upload-report	Upload diagnostic report or document
GET	/ehr	Retrieve electronic health record
GET	/notifications	View pending alerts and reminders

1.5.3 Doctor APIs

Base path: /api/v1/doctor

Table 1.7: Doctor API Endpoints

Method	Endpoint	Purpose
GET	/appointments	View scheduled appointments
GET	/patients/{id}	Retrieve individual patient details
POST	/consultation	Record new consultation entry
POST	/prescription	Generate patient prescription
PUT	/ehr-update	Update patient EHR record

1.5.4 Appointment APIs

Base path: /api/v1/appointments

Table 1.8: Appointment API Endpoints

Method	Endpoint	Purpose
POST	/book	Book a new appointment
PUT	/reschedule/{id}	Reschedule an existing appointment
DELETE	/cancel/{id}	Cancel an appointment
GET	/history	Retrieve appointment history

1.5.5 API Access Control

Access to each API group is governed by role-based permissions. The table below defines the level of access granted to each user role across all API groups.

Table 1.9: API Access Control Matrix (RBAC)

API Group	Patient	Doctor	Admin
Authentication APIs	Allowed	Allowed	Allowed
Patient APIs	Own records only	Assigned patients	Monitor only
Doctor APIs	No access	Allowed	Monitor only
Appointment APIs	Own appointments	Assigned appointments	Full access
EHR APIs	Own EHR	Assigned patients	Restricted
Admin APIs	No access	No access	Allowed

1.6 Security Design

All API requests traverse a multi-stage security pipeline. Each stage enforces an additional layer of control before data reaches persistent storage.

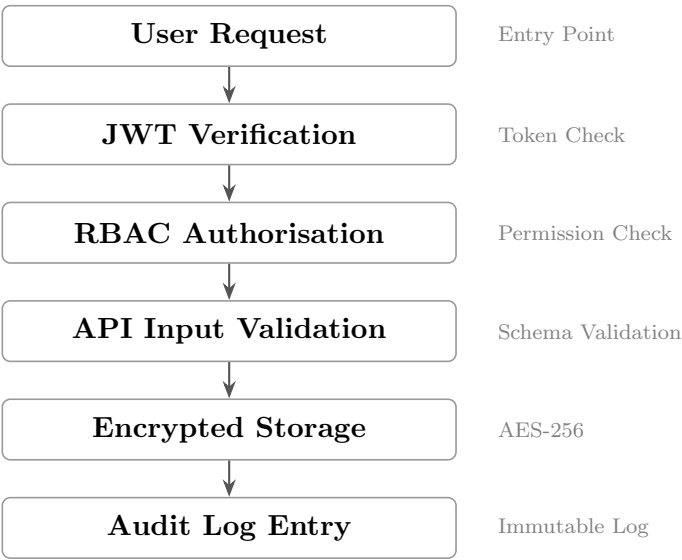


Figure 1.3: Security Enforcement Pipeline

1.7 Fallback Design

The system implements a graceful degradation strategy. In the event of any component failure, an automatic fallback mechanism ensures continuity of care.

Table 1.10: Failure Scenarios and Fallback Actions

Failure Scenario	Fallback Action
No Internet Connectivity	Data persisted locally via IndexedDB; synced on reconnection
Video Consultation Failure	Automatic downgrade to audio-only consultation
Audio Consultation Failure	Automatic downgrade to text chat consultation
AI Triage Engine Failure	Rule-based decision tree activated as substitute
OCR Processing Failure	Patient prompted for manual document upload
Doctor Unavailability	Appointment automatically reassigned to next available doctor

The consultation channel fallback hierarchy is illustrated below.

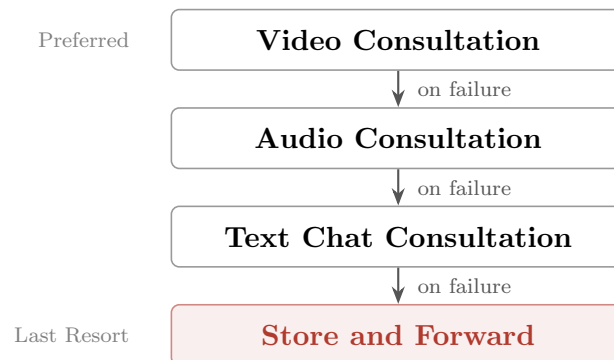


Figure 1.4: Consultation Fallback Hierarchy

1.7.1 Offline Synchronization Flow

A key requirement for rural and low-connectivity deployments is the ability to continue recording patient data without an active internet connection. The offline synchronization module handles this transparently, queuing all writes locally and replaying them once connectivity is restored.

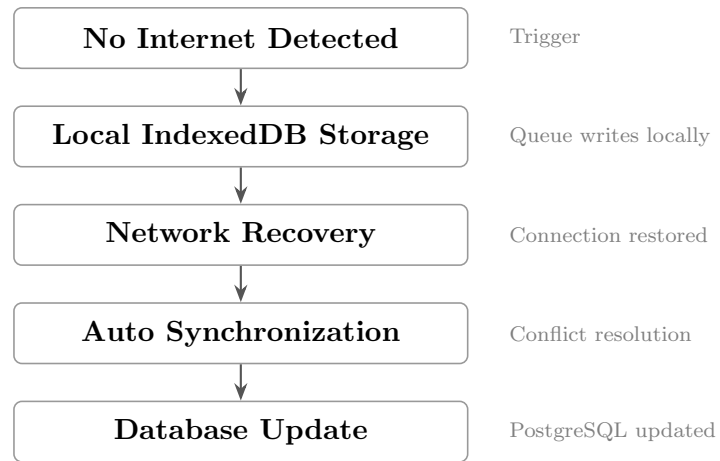


Figure 1.5: Offline Synchronization Flow